

Technical Design: Multi-Source Candidate Data Transformer

One canonical, explainable ATS profile from messy structured and unstructured sources

Pipeline

Upload/source detect -> parse -> extract -> LLM refine -> normalize -> merge -> confidence/provenance -> project -> validate -> dynamic UI

Extraction and normalization

- Source inference maps CSV to job-board, JSON to ATS, recruiter notes to manual, and PDF/DOCX/text resumes to resume extraction.
- Parsers convert files to typed parsed content; extractors produce PartialCandidate records for resumes, notes, CSV rows, and ATS blobs.
- Resume extraction uses section boundaries plus date-bearing job headers, so bullets remain job descriptions rather than bogus employers.
- Normalizers canonicalize email, E.164 phone, URL, ISO-ish dates, location country, company, degree, experience dates, and skill names.
- LLM refinement is bounded to source evidence for ambiguous skills, employers, headlines, and semantic confidence; deterministic output remains the fallback.

Merge, provenance, and confidence

- Identity grouping uses stable source records and deterministic IDs to collect same-candidate partials before canonical build.
- Per-field merge plans choose strategies: scalar for names/headline, contact/social dedupe, array skill union, and typed education/experience merges.
- Conflict winners prefer meaningful, complete values and source priority; discarded values are retained in merge provenance.
- Provenance records carry field path, source record, extractor/normalizer/merge method, winner IDs, and discarded alternatives.
- Confidence combines source reliability, agreement, completeness, validation, field weights, and optional LLM-grounded hybrid adjustments.

Canonical schema and normalized formats

Area	Canonical representation
Identity/contact	candidate id, full name, headline, emails, E.164 phones, location {city, region, ISO alpha-2 country}, links
Profile signals	years experience, canonical skills with per-skill confidence and sources, summary/tags when present
History	experience {company, title, start/end YYYY-MM, summary}; education {institution, degree, field, end_year}
Trust metadata	field provenance, field confidence, overall confidence, optional LLM decisions/warnings/explanations

Runtime projection

The engine always builds the internal canonical profile first, then projects it. A runtime config may select fields, rename from canonical paths, request per-field formatting, include confidence/provenance/source records, and choose missing value policy: omit, null, or error. Bad or empty config is parsed through parseProjectionConfigOrDefault and safely falls back to the required default assignment schema.

Edge cases and deliberate scope

Handled

- Missing/garbage source: parser/extractor errors are isolated; unknown values become null/omitted by policy, not invented.
- Malformed projection config: warning plus default output schema, so the demo/API does not fail.
- Resume line wrapping: date-header parser prevents bullets and continuations from becoming employers.
- Non deterministic information handling and sparse data handling using LLM inference for refining before processing.

Deliberately scoped under time pressure

- No broad web crawling of LinkedIn/GitHub; only provided files/URLs and available public fields are in scope.
- OCR for scanned/image-only resumes is not implemented; text-bearing PDF/DOCX is the target.
- Human review UI for merge conflicts is omitted; provenance makes automated decisions auditable.
- Frontend is an inspection console, not a full recruiter workflow or ATS replacement.

Why this design is defensible

Separation of concerns keeps the engine explainable: extraction gathers evidence, normalization standardizes values, merge resolves conflicts, confidence/provenance records trust, and projection reshapes output without touching core logic. The dynamic frontend mirrors this by rendering arbitrary projected JSON while preserving raw output for evaluation.